Topology Optimization of structures

Final Review on 23/06/2022

Guided by
Dr. Arun M
Associate Professor
PSG Institute of Technology and Applied Research

Aim

To make an python program for size optimization and topology optimization.

Objective

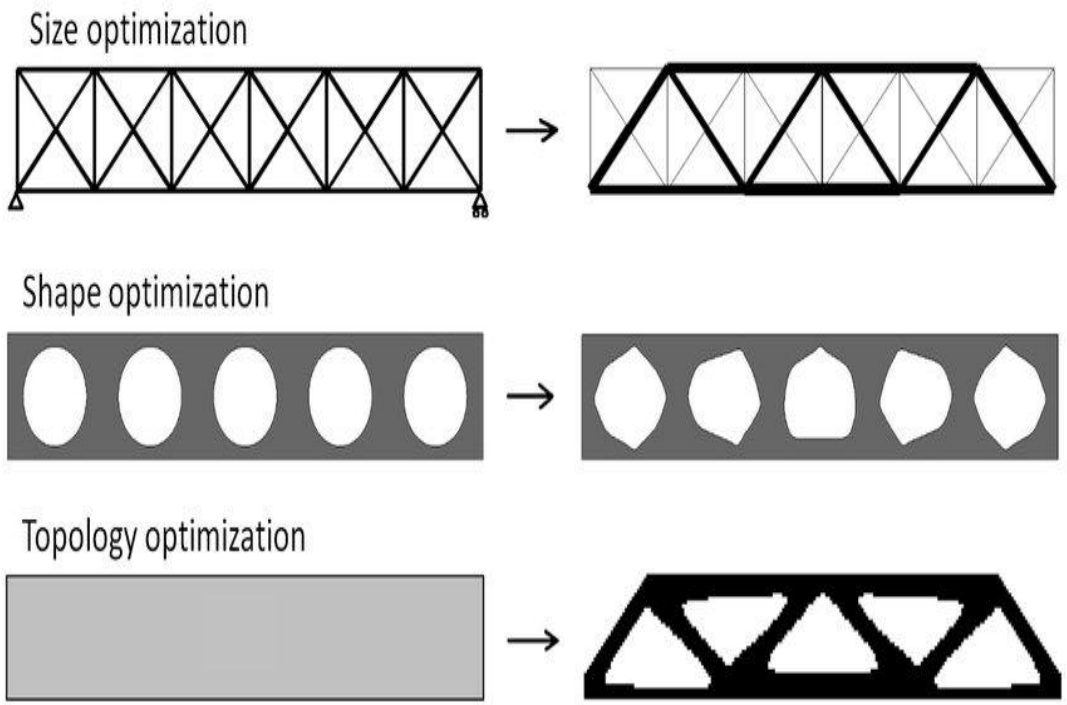
- ✓ Make an algorithm for size and topology optimization.
- ✓ Study and develop code for Finite Element Model.
- ✓ Develop code for size and topology optimization.

Topology Optimization of Structures

1 Size optimization

2 Shape optimization

3 Topology optimization



Topology optimization

Topology

Mathematics field properties
of invariant spaces
continuous deformation

+

Optimization

making design
as effective as possible

→

Topology optimization

Mathematical method
optimizes material,
design space,
set of loads and
constraints maximize the
performance

Literature review
Topology optimization

Objective function – Minimization of **strain energy**

$$- \min \int_0^l \frac{1}{2} AEU'^2 \mathrm{dx}$$

Constrain 1 $- \int Adx - \mathrm{V} \leq 0$

Constrain 2 $- \min \int_0^l \frac{1}{2} AEU'^2 - \mathrm{P(x)U(x)}$

Literature review
Optimality criteria method

It is a **convergence** based **iterative** method to do topology optimization

On solving above functional we get Λ(lagrange multiplier)

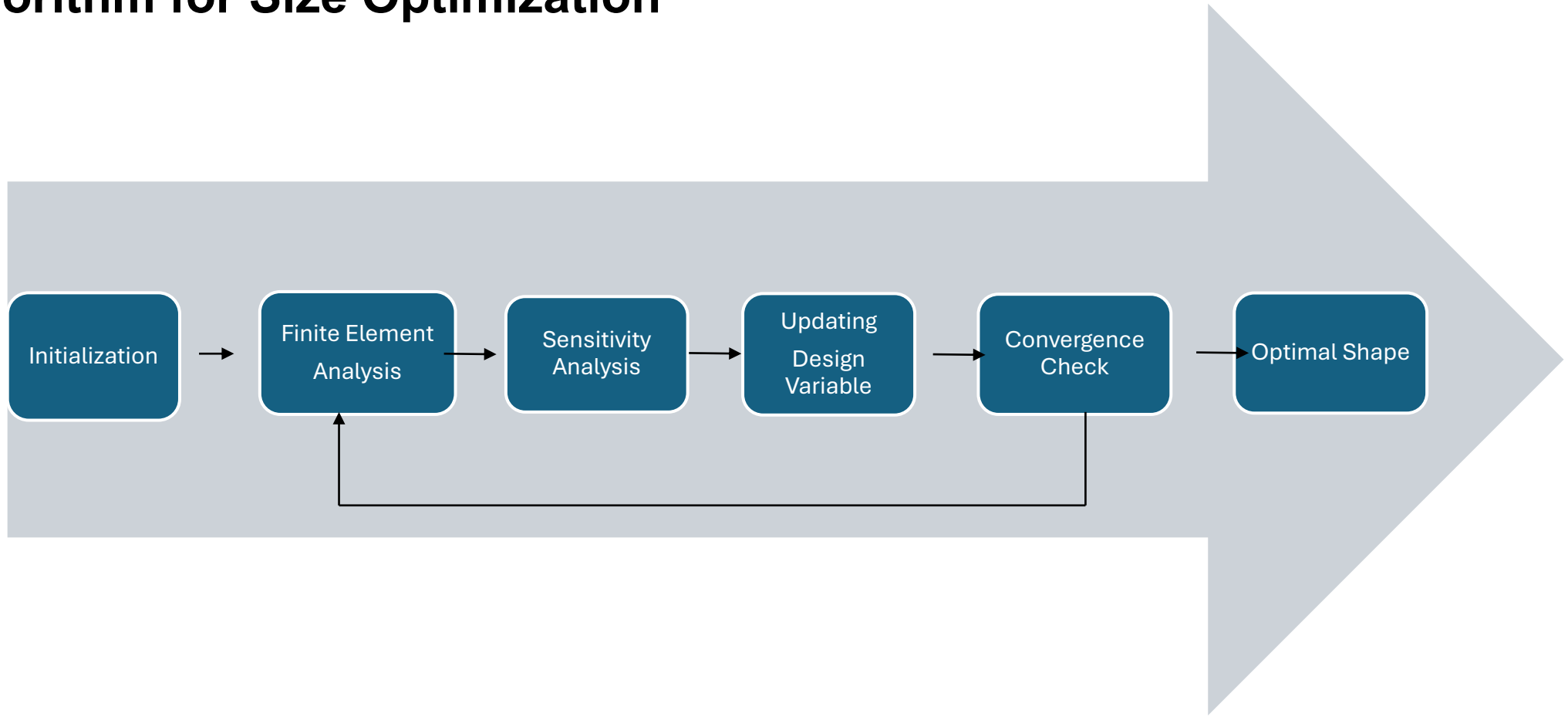
$$\Lambda = \frac{EU'^2}{2}$$

Based on this topology optimization is done.

Work Flow

Literature review
Algorithm
Finite Element Analysis
Topology Optimization

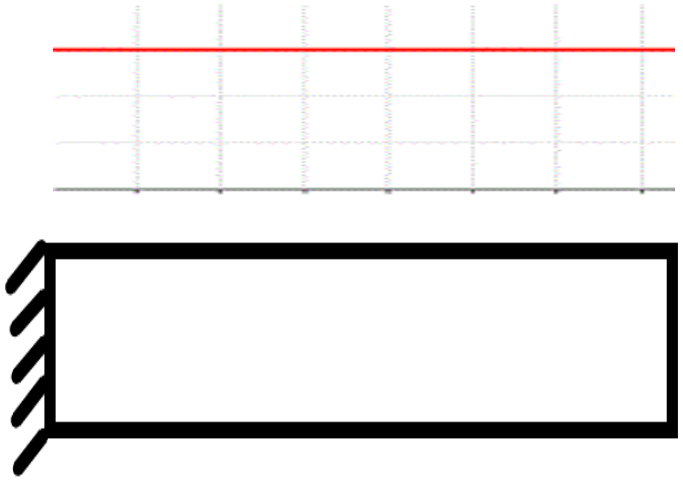
Algorithm for Size Optimization



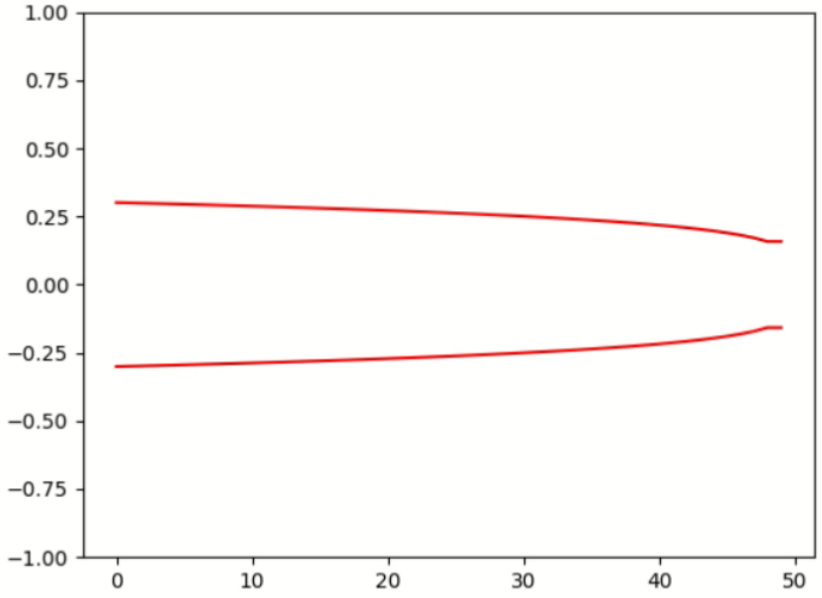
1-Dimensional optimization – Python code

```python
import matplotlib.pyplot as plt
import numpy as np
import math
import ast
filename=input("Enter the file name")
nomember=50
E=210
Vol=2.5
length=10
area=vol/length
lengthsegment=length/nomember
depth=1
gcl=[];grl=[]
#endconditions matrix
constrain=[1,2,3]
for i in range(4,(nomember+1)*3+1):
        gcl.append(i)
        grl.append(i)
gc=gcl+constrain
gr=grl+constrain
#loadmatrix
uinitial=[]
load=[]
for i in range(1,(nomember+1)*3):
        if((i+2)%3==0):#i==(nomember*3)+1)
                uinitial.append([i,math.cos(math.radians(i*2.4))])
                load.append(math.cos(math.radians(i*2.4)))
                #uinitial.append([i,1])
                #load.append(1)
        else:
                uinitial.append([i,0])
#uinitial[(25*3)]=[(25*3),10]
#print(uinitial)
uinitial[0]=[1,0]
uinitial[1]=[2,0]
uinitial[2]=[3,0]
ugiven=[]
for i in gc:
        for j in uinitial:
                if(i==j[0]):
                        ugiven.append(j[1])
                else:
                        pass
bl=[]
for i in range(nomember):
    bl.append(vol/length/depth)

#breath will be varying along length
lambdaa=0
maxis=[]
for k in range(nomember-1):
    maxis.append(k/nomember*length)
load.pop()
load.pop()
plt.plot(maxis,load)
plt.ylim(min((load)*5),max(load)*5)
plt.show()
savebreath=[]
for j in range(20):
        FElength=length/nomember
        #area=depth*breath(breath is variable)
        alpha=1
        beta=1
        GSM=[]
        LGSM=[]
        for i in range(1,nomember+1):
                MSM=[]
                a11=(depth*bl[i-1]*E/FElength*alpha*alpha)+(12*E*(bl[i-1]*depth**3)/12/FElength/FElength/FElength*beta*beta)
                a12=(depth*bl[i-1]*E/FElength-12*E*(bl[i-1]*depth**3)/12/FElength/FElength/FElength)*(alpha*beta)
                a22=(depth*bl[i-1]*E/FElength*beta*beta)+(12*E*(bl[i-1]*depth**3)/12/FElength/FElength/FElength*alpha*alpha)
                a13=6*E*(bl[i-1]*depth**3)/12/FElength/FElength*beta
                a23=6*E*(bl[i-1]*depth**3)/12/FElength/FElength*alpha
                a33=2*E*(bl[i-1]*depth**3)/12/FElength
                ma11=(depth*bl[i-1]*E/FElength*alpha)
                ma21=12*(E*(bl[i-1]*depth**3)/12/FElength/FElength/FElength)*beta
                ma31=6*(E*(bl[i-1]*depth**3)/12/FElength/FElength)*beta
                ma32=6*(E*(bl[i-1]*depth**3)/12/FElength/FElength)*alpha
                ma12=(depth*bl[i-1]*E/FElength*beta)
                ma22=12*(E*(bl[i-1]*depth**3)/12/FElength/FElength/FElength)*alpha
                ma23=6*(E*(bl[i-1]*depth**3)/12/FElength/FElength)

                MSM=[[a11,a12,-a13,-a11,-a12,-a13],[a12,a22,a23,-a12,-a22,a23],[-a13,a23,a33,a13,-a23,a331],
                        [-a11,-a12,a13,a11,a12,a13],[-a12,-a22,-a23,a12,a22,-a23],[-a13,a23,a331,a13,-a23,a331],[(i*3)-2,(i*3)-1,i*3,(i*3)+1,(i*3)+2,(i*3)+3]]
                LMSM=[[ma11,ma12,0,-ma11,-ma12,0],[-ma21,ma22,ma23,ma21,-ma22,ma23],[-ma31,ma32,a33,ma31,-ma32,a331],
                        [-ma11,-ma12,0,ma11,ma12,0],[ma21,-ma22,-ma23,-ma21,ma22,-ma23],[-ma31,ma32,a331,ma31,-ma32,a333]]
                GSM.append(MSM)
                LGSM.append(LMSM)
        Cl=[]
        for Mc in gc:
            Rl=[]
            for Mr in gr:
                y=0
                for mn in range(0,nomember):
                    for mr in range(0,6):
                        if(GSM[mn][6][mr]==Mr):
                            for mc in range(0,6):
                                if(GSM[mn][6][mc]==Mc):
                                    x=GSM[mn][mc][mr]
                                    y=y+x
                Rl.append(y)
            Cl.append(Rl)
        ugiven1=ugiven[0:len(grl)]
        multmat=[]
        for i in Cl:
                multmat.append(i[0:len(grl)])
        multmat=multmat[0:len(grl)]

        ugiven2=np.dot((np.linalg.inv(multmat)),ugiven1)
        u=ugiven2[0:len(ugiven2):3]
        lambdaa=0
        for i in range(nomember-1):
                if(i==0):
                        lambdaa=lambdaa+depth*bl[i]*lengthsegment*(E*((u[i])/(length/nomember))**2)**0.1
                else:
                        lambdaa=lambdaa+depth*bl[i]*lengthsegment*(E*((u[i]-u[i-1])/(length/nomember))**2)**0.1
        lambdaa=(lambdaa/vol)**(1/0.1)
        check=0
        for i in range(nomember):
                if(i==0):
                        breath=bl[i]*(E*((u[i])/(length/nomember))**2/lambdaa)**0.1
                else:
                        breath=bl[i]*(E*((u[i]-u[i-1])/(length/nomember))**2/lambdaa)**0.1
                        if(((E*((u[i]-u[i-1])/(length/nomember))**2/lambdaa))>0.9 and ((E*((u[i]-u[i-1])/(length/nomember))**2/lambdaa))<1.1):
                                check=1+check
                bl[i]=breath
        bl[nomember-1]=bl[nomember-2]
        print(lambdaa,check)
        c=bl[0:nomember-1]
        """plt.plot(maxis,c)
        plt.ylim(0,max(bl)*5)
        plt.show()"""
        savebreath.append(list(bl))
        if(check>=40):
                break
outputlist=[savebreath]
out_file = open(str(filename)+".txt", "w")
json.dump(outputlist, out_file,indent =0)
out_file.close()
plt.plot(maxis,c)
plt.ylim(0,max(bl)*5)
plt.show()

r=...
plt.ylim(0,max(bl)*5)
plt.show()
```

1-Dimensional bar optimization



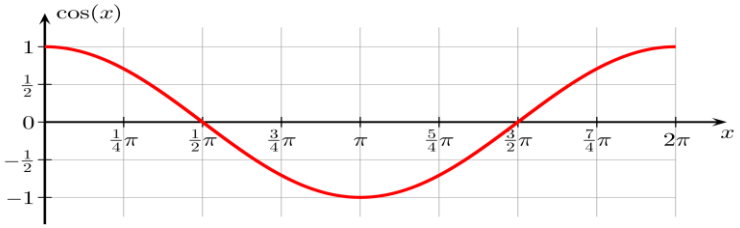
Uniform axial load is applied throughout the bar



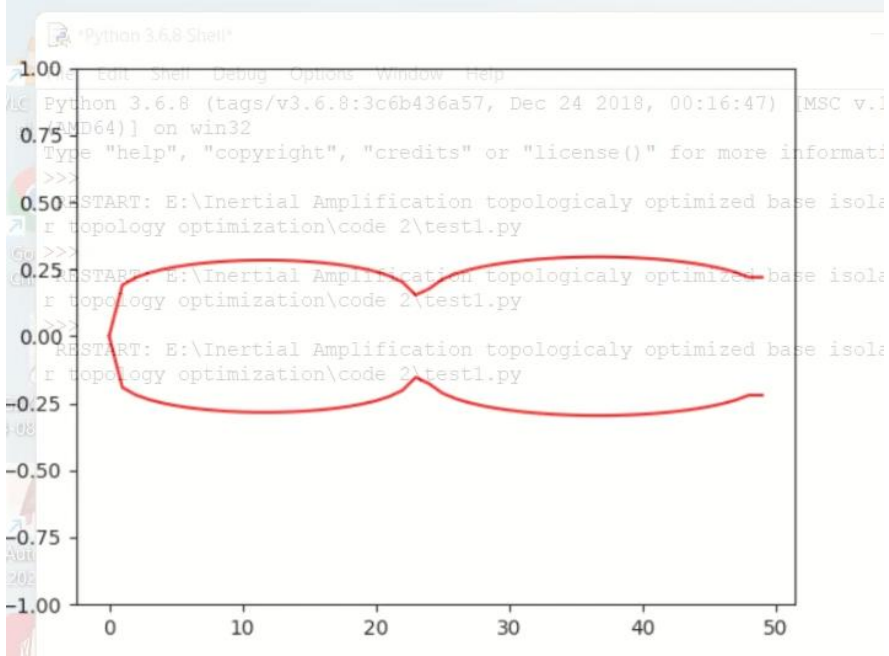
Optimized shape

Optimized shape is obtained in such a way that all **elements** have **equal stress**

1-Dimensional bar optimization



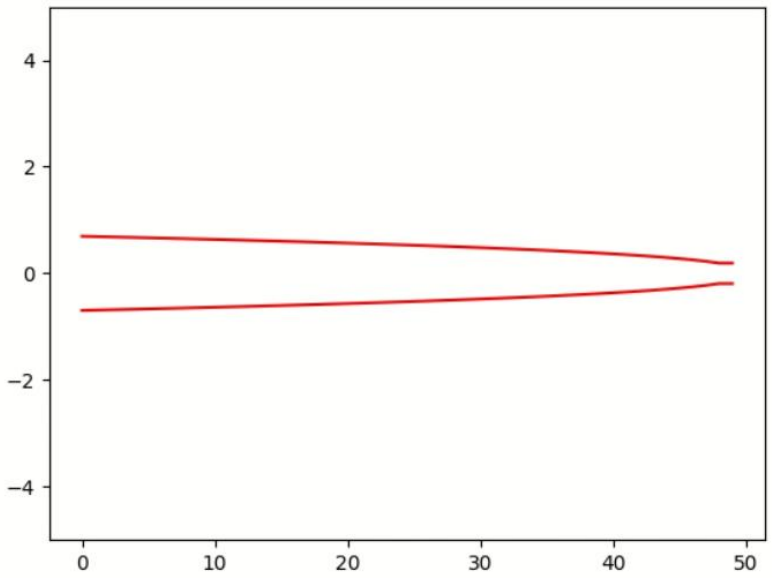
Axial load is applied in form of cosine wave



Optimized shape

1- Dimensional beam optimization

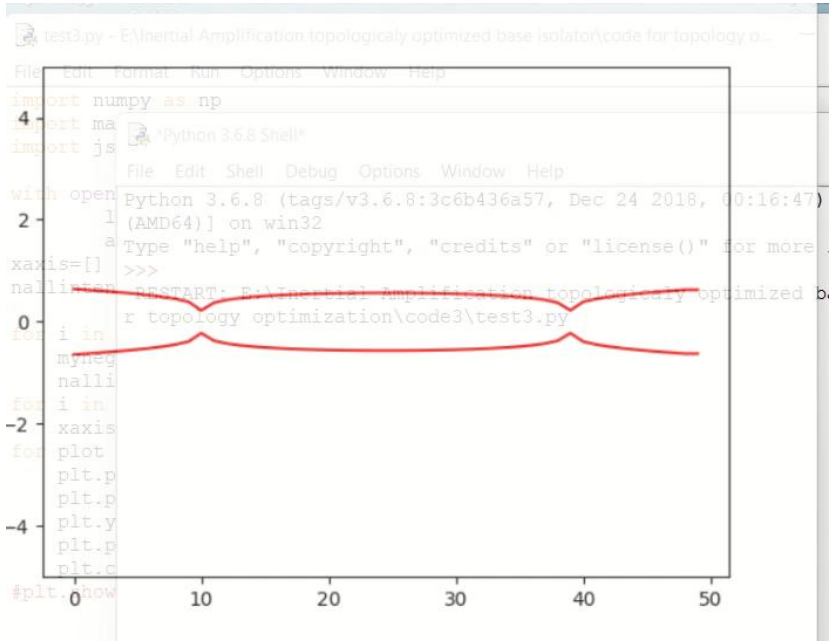
Cantilever beam with Uniformly distributed load



Optimized shape

1- Dimensional beam optimization

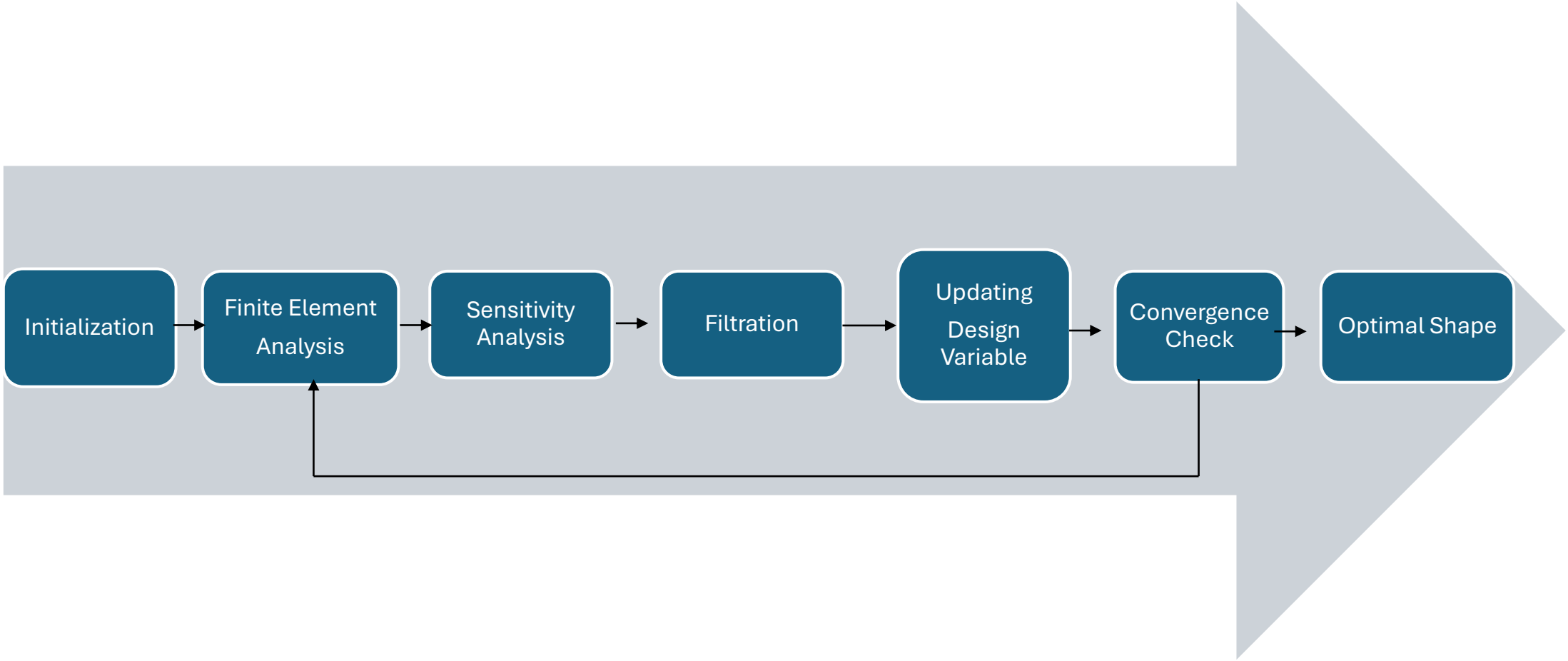
Fixed beam with Uniformly distributed load



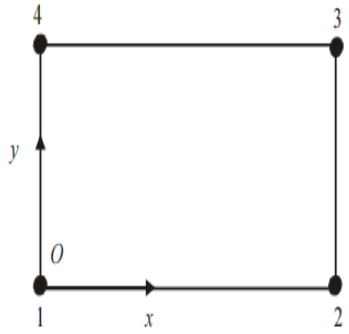
Optimized shape

Optimized shape matches with shape of Bending moment diagram

Algorithm for Topology optimization



Finite element analysis



Four noded square element

$$[k] = \int [B]^T [D][B]$$

[B]- Strain displacement relationship

[D]- Stress Strain relationship

Strain displacement matrix

Shape Functions

$N_1 = \left(1 - \frac{x}{l} - \frac{y}{h} + \frac{xy}{hl}\right)$

$N_2 = \left(\frac{x}{l} - \frac{xy}{lh}\right)$

$N_3 = \left(\frac{xy}{lh}\right)$

$N_4 = \left(\frac{y}{h} - \frac{xy}{lh}\right)$

$$[B] = \begin{bmatrix} -\frac{1}{l}+\frac{y}{hl} & 0 & \frac{1}{l}-\frac{y}{hl} & 0 & \frac{y}{hl} & 0 & -\frac{y}{hl} & 0 \\ 0 & -\frac{1}{l}+\frac{x}{hl} & 0 & -\frac{x}{hl} & 0 & \frac{x}{hl} & 0 & \frac{1}{l}-\frac{x}{hl} \\ -\frac{1}{l}+\frac{x}{hl} & -\frac{1}{l}+\frac{y}{hl} & -\frac{x}{hl} & \frac{1}{l}-\frac{y}{hl} & \frac{x}{hl} & \frac{y}{hl} & \frac{1}{l}-\frac{x}{hl} & -\frac{y}{hl} \end{bmatrix}$$

Stress Strain matrix

$$[D] = \begin{bmatrix} 1 & u & 0 \\ u & 1 & 0 \\ 0 & 0 & \frac{1-u}{2} \end{bmatrix}$$

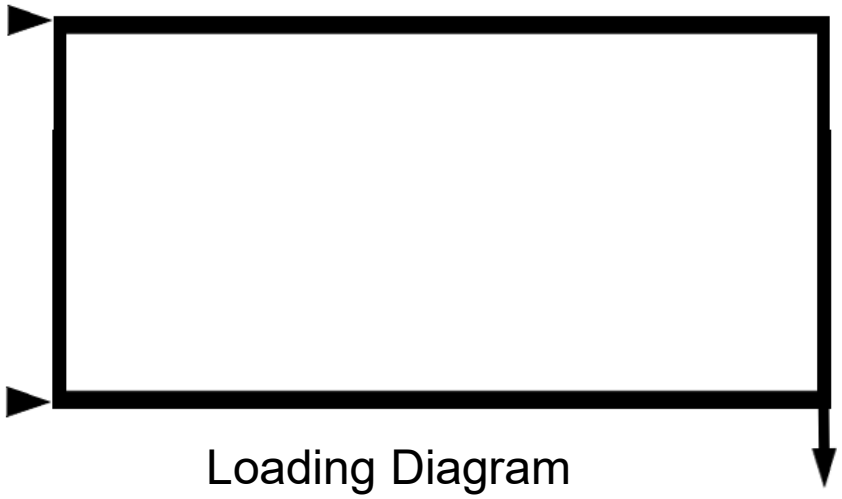
Here this matrix is formed by assuming plane stress condition. This assumption is valid because we are using thin elements.

Element Stiffness matrix

$$[K]=\frac{E}{(1-\mu^2)}\begin{bmatrix} \frac{(3-\mu)}{6} & \frac{(1+\mu)}{8} & -\frac{(3+\mu)}{12} & \frac{(-1+3\mu)}{8} & \frac{(-3+\mu)}{12} & -\frac{(1+\mu)}{8} & \frac{(\mu)}{6} & \frac{(1-3\mu)}{8} \\ \frac{(1+\mu)}{8} & \frac{(3-\mu)}{6} & \frac{(1-3\mu)}{8} & \frac{(\mu)}{6} & -\frac{(1+\mu)}{8} & \frac{(-3+\mu)}{12} & \frac{(-1+3\mu)}{8} & -\frac{(3+\mu)}{12} \\ -\frac{(3+\mu)}{12} & \frac{(1-3\mu)}{8} & \frac{(3-\mu)}{6} & -\frac{(1+\mu)}{8} & \frac{(\mu)}{6} & \frac{(-1+3\mu)}{8} & \frac{(-3+\mu)}{12} & \frac{(1+\mu)}{8} \\ \frac{(-1+3\mu)}{8} & \frac{(\mu)}{6} & -\frac{(1+\mu)}{8} & \frac{(3-\mu)}{6} & \frac{(1-3\mu)}{8} & -\frac{(3+\mu)}{12} & \frac{(1+\mu)}{8} & \frac{(-3+\mu)}{12} \\ \frac{(-3+\mu)}{12} & -\frac{(1+\mu)}{8} & \frac{(\mu)}{6} & \frac{(1-3\mu)}{8} & \frac{(3-\mu)}{6} & \frac{(1+\mu)}{8} & -\frac{(3+\mu)}{12} & \frac{(-1+3\mu)}{8} \\ -\frac{(1+\mu)}{8} & \frac{(-3+\mu)}{12} & \frac{(-1+3\mu)}{8} & -\frac{(3+\mu)}{12} & \frac{(1+\mu)}{8} & \frac{(3-\mu)}{6} & \frac{(1-3\mu)}{8} & \frac{(\mu)}{6} \\ \frac{(\mu)}{6} & \frac{(-1+3\mu)}{8} & \frac{(-3+\mu)}{12} & \frac{(1+\mu)}{8} & -\frac{(3+\mu)}{12} & \frac{(1-3\mu)}{8} & \frac{(3-\mu)}{6} & -\frac{(1+\mu)}{8} \\ \frac{(1-3\mu)}{8} & -\frac{(3+\mu)}{12} & \frac{(1+\mu)}{8} & \frac{(-3+\mu)}{12} & \frac{(-1+3\mu)}{8} & \frac{(\mu)}{6} & -\frac{(1+\mu)}{8} & \frac{(3-\mu)}{6} \end{bmatrix}$$

Topology optimization – Python code

Cantilever Beam

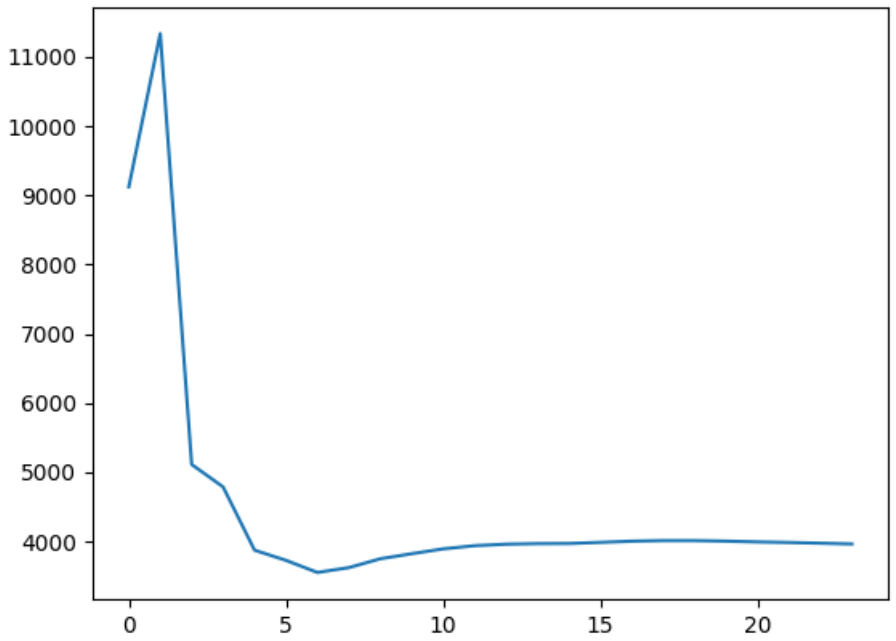


Loading Diagram

Finite Element Analysis model

Length = 3000 mm
Depth = 1500 mm
Load = 10000 N
Youngs Modulus = 200000 N/mm^2
Poisson Ratio = 0.3

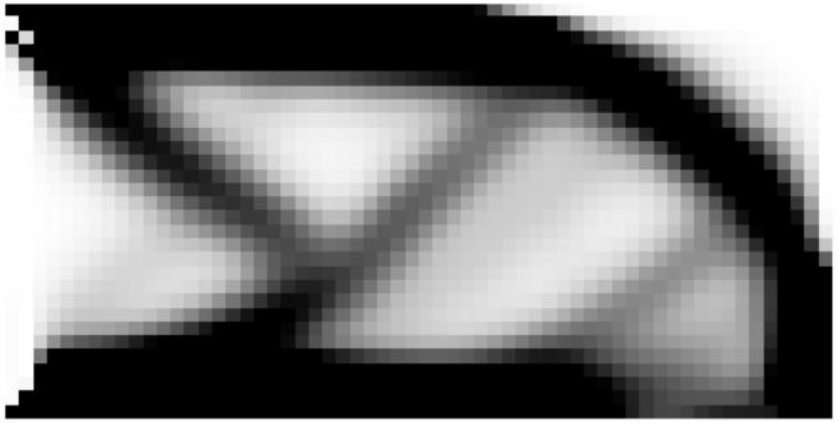
Strain Energy



- X axis – Iterations
- Y axis - Strain energy in N-mm
- The strain energy of model before optimization was 9121.78 N-mm
- The strain energy of system after optimization was 3964.92 N-mm
- The strain energy has reduced by 56.53 %

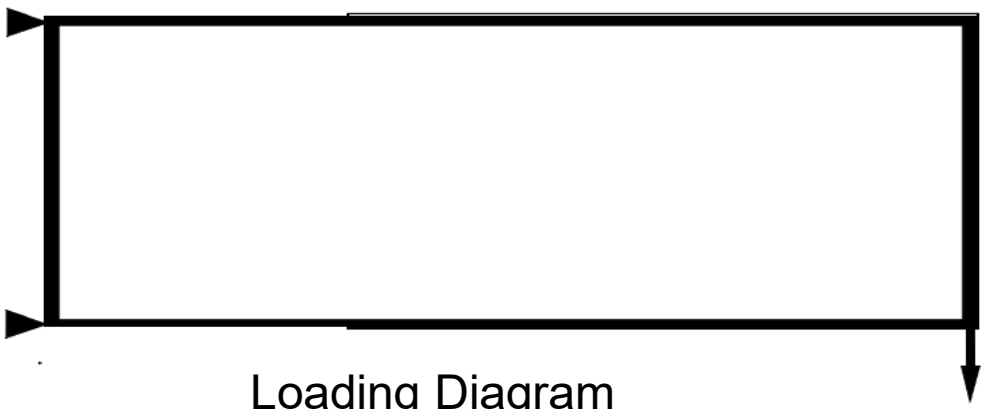
Optimization

Optimized shape without filter



Optimized shape with filter

Cantilever Beam

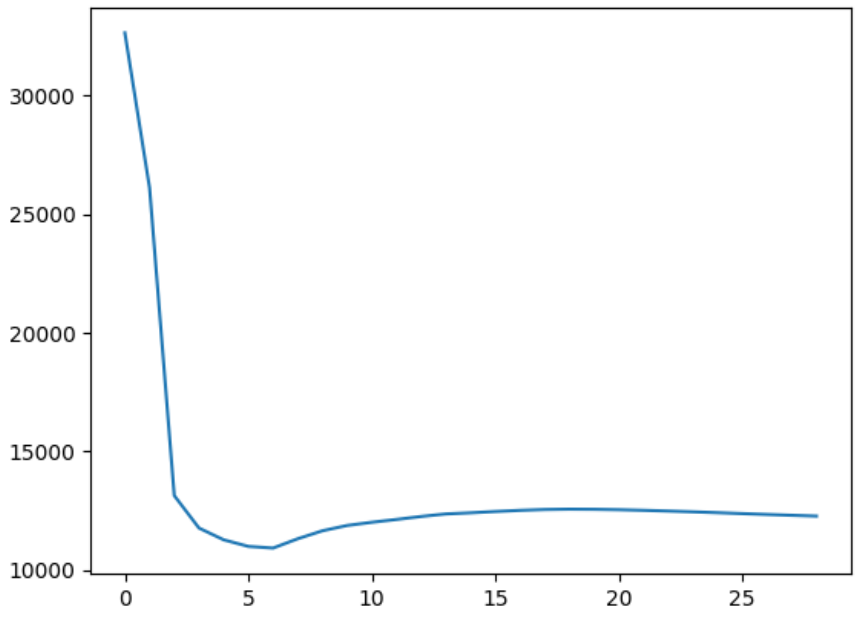


Loading Diagram

Length = 2250 mm
Depth = 750 mm
Load = 10000 N
Youngs Modulus = 200000 N/mm^2
Poisson Ratio = 0.3

Finite Element Analysis model

Strain Energy

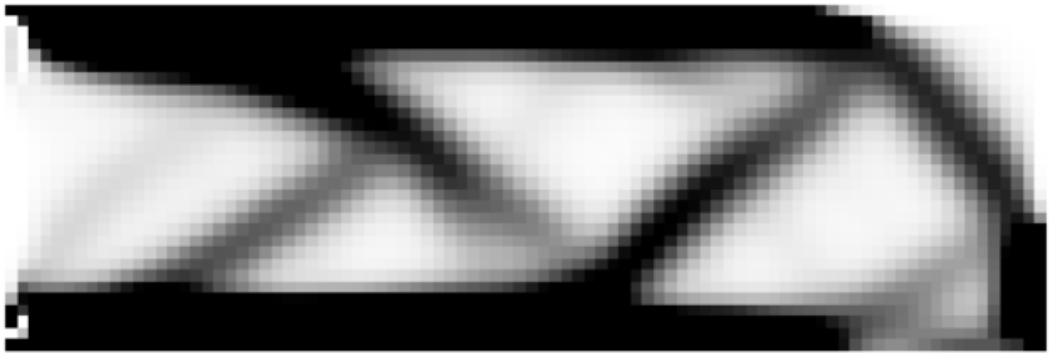


- X axis – Iterations
- Y axis - Strain energy in N-mm
- The strain energy of model before optimization was 32641 N-mm
- The strain energy of system after optimization was 12276 N-mm
- The strain energy has reduced by 62.39 %

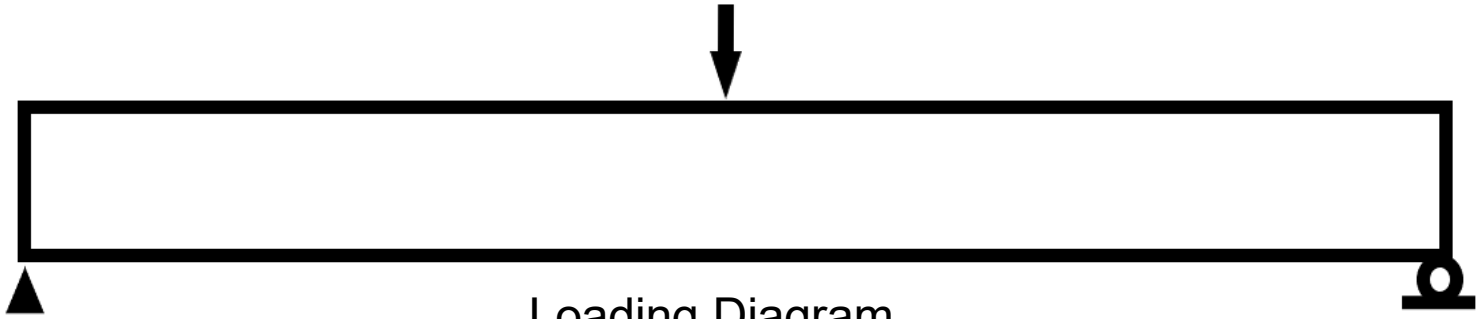
Optimization

Optimized shape without filter

Optimized shape with filter



Simply supported Beam

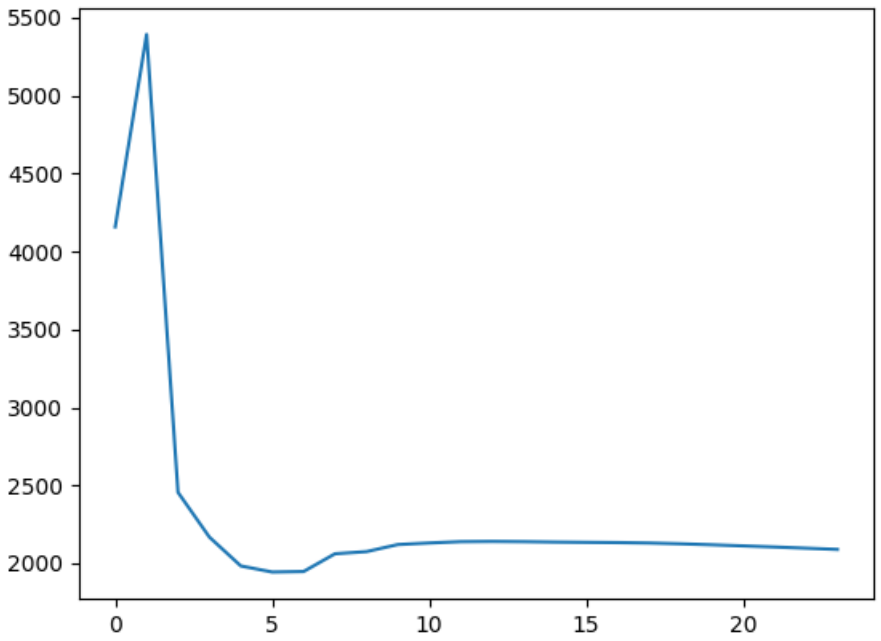


Loading Diagram

Length = 4000 mm
Depth = 1000 mm
Load = 10000 N
Youngs Modulus = 200000 N/mm^2
Poisson Ratio = 0.3

Finite Element Analysis model

Strain Energy



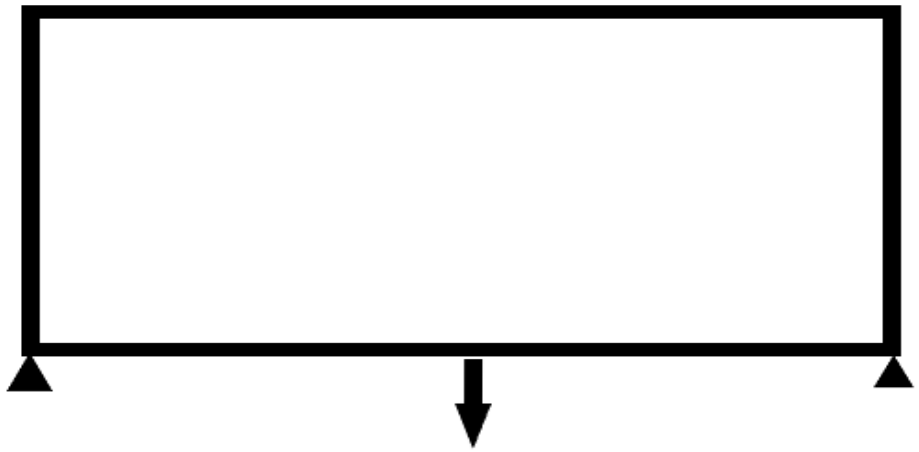
- X axis – Iterations
- Y axis - Strain energy in N-mm
- The strain energy of model before optimization was 4157.27 N-mm
- The strain energy of system after optimization was 2089.38 N-mm
- The strain energy has reduced by 49.74 %

Optimization

Optimized shape without filter

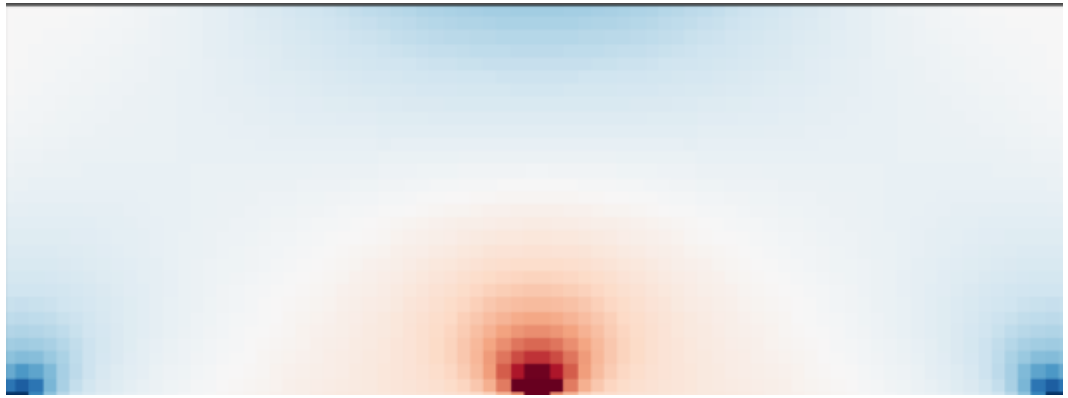
Optimized shape with filter

Simply supported Beam with both ends hinged



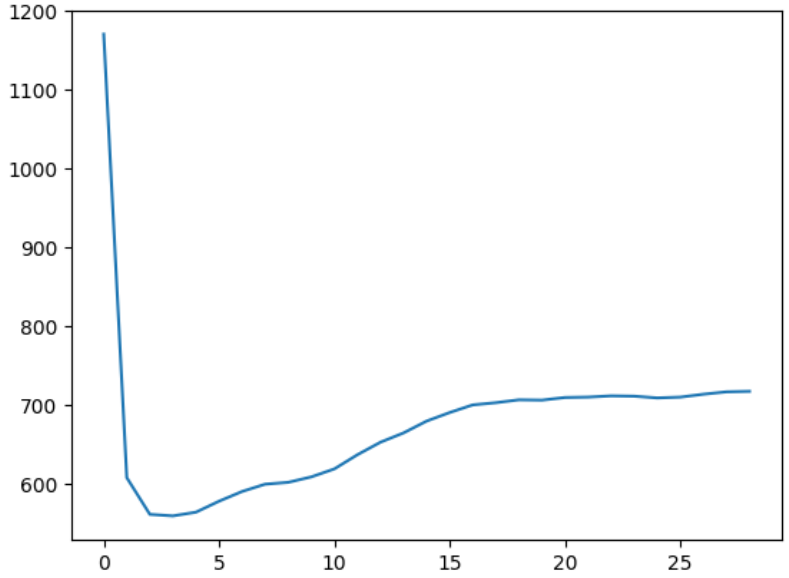
Loading Diagram

Length = 2250 mm
Depth = 750mm
Load = 10000 N
Youngs Modulus = 200000 N/mm^2
Poisson Ratio = 0.3



Finite Element Analysis model

Strain Energy



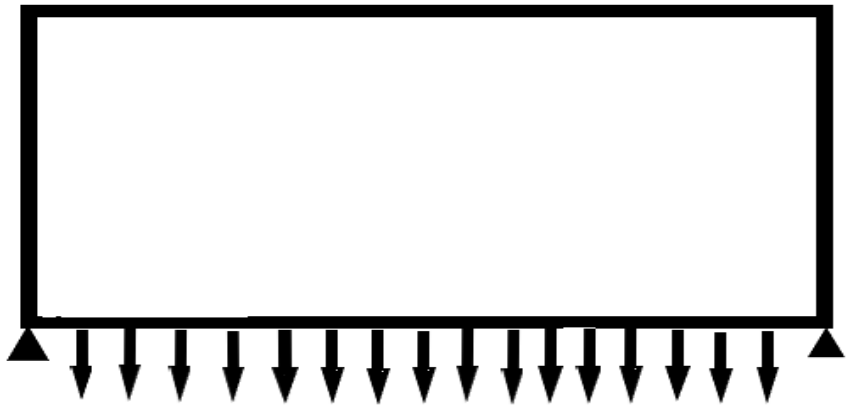
- X axis – Iterations
- Y axis - Strain energy in N-mm
- The strain energy of model before optimization was 1170.53 N-mm
- The strain energy of system after optimization was 909.27 N-mm
- The strain energy has reduced by 22.33 %

Optimization

Optimized shape without filter

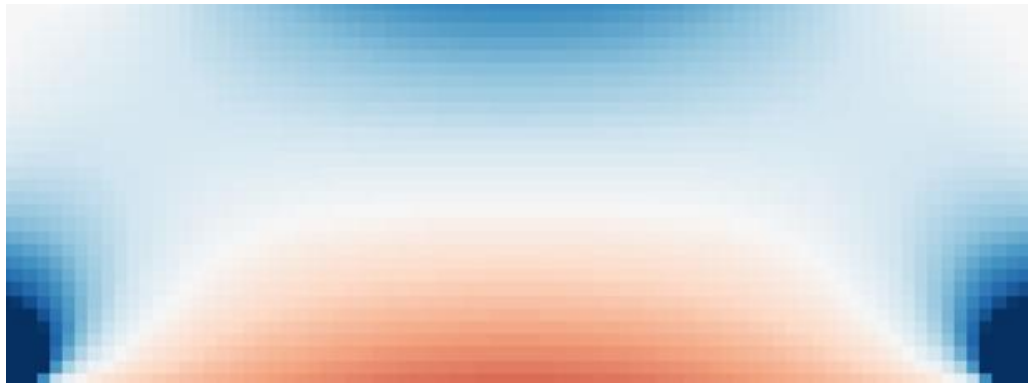
Optimized shape with filter

Simply supported Beam with both ends hinged



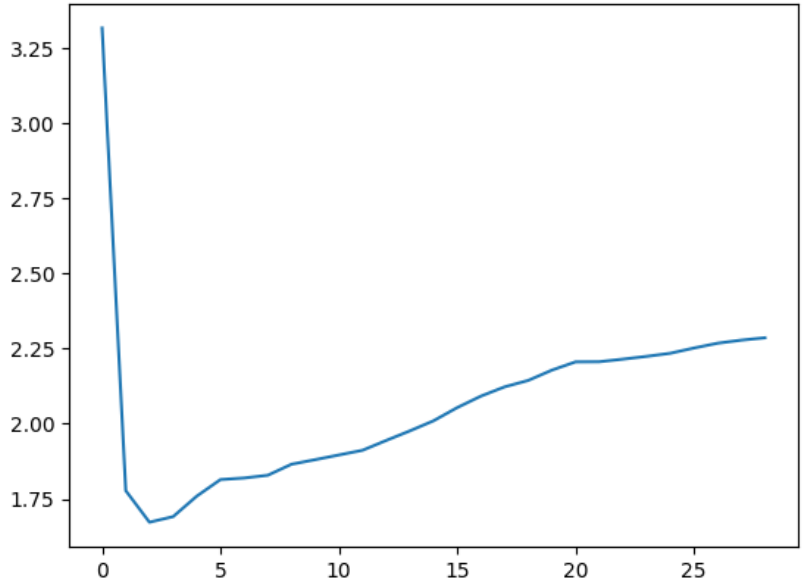
Loading Diagram

Length = 2250 mm
Depth = 750 mm
Load = 0.4 N/mm
Youngs Modulus = 200000 N/mm^2
Poisson Ratio = 0.3



Finite Element Analysis model

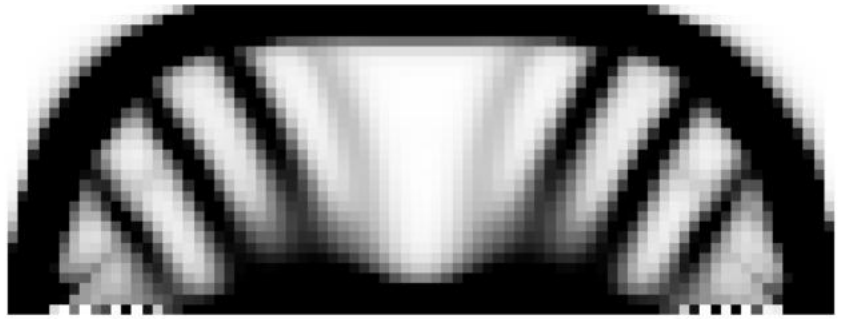
Strain Energy



- X axis – Iterations
- Y axis - Strain energy in N-mm
- The strain energy of model before optimization was 3.317 N-mm
- The strain energy of system after optimization was 2.286 N-mm
- The strain energy has reduced by 31.08 %

Optimization

Optimized shape without filter



Optimized shape with filter

Conclusion

- ✓ The optimality criteria condition created using minimum strain energy principal successfully optimizes the structure.
- ✓ The optimized shape for Simply supported stores 49.74% less strain energy and cantilever beam stores 62.39% less strain energy.
- ✓ From above five examples, optimized shapes either forms arch or truss like structures, where there will be direct axial stress be developed.
- ✓ In the places where topology optimization is not possible we can go with above inference to get stiffest structure.

References

- M.P. Bendose: “Optimal shape design as a material distribution problem”
- Ole Sigmun: “Morphology-based black and white filters for topology optimization.”